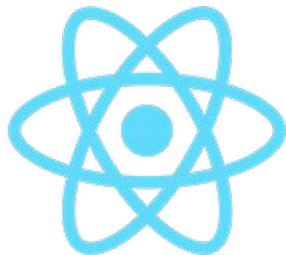


REACT

웹 및 사용자 인터페이스를 위한 라이브러리



리액트?

- 리액트는 SPA(Single Page Application)을 위한 자바스크립트 라이브러리
- 2013년 페이스북(현 메타)에서 공개
- 웹 UI를 작성하기 위한 목적으로 만들어졌으며 순수하게 자바스크립트만을 이용해서 만들 수 있고,
- UI영역의 재사용을 위해 '컴포넌트'라는 작은 단위 기반으로 개발
- 이렇게 만들어진 여러 개의 컴포넌트를 조합하여 UI를 작성

SPA(Single Page Application)는 서버로부터 완전한 새로운 페이지를 불러오지 않고

현재의 페이지를 동적으로 다시 작성함으로써 사용자와 소통하는 웹 애플리케이션이나 웹사이트를 말한다.

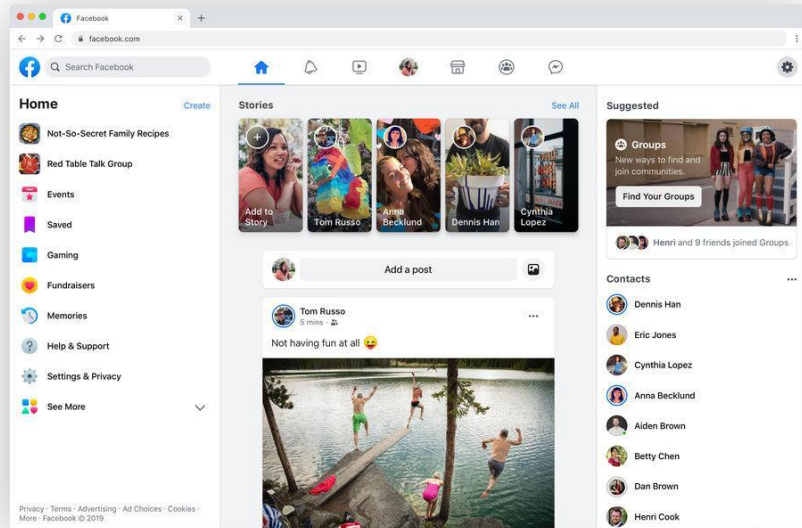
-위키백과-

리액트의 필요성

- 동적인 변화가 많은 웹사이트 구현
- 페이스북과 같은 웹의 탭에 의한 [화면일부] 변경을 보다 부드럽게 하기 위해 - 마치 앱처럼
 - 그래서 요즘 웹은 웹페이지 보다는 웹 애플리케이션이라고 부르게 됨.

리액트팀에서 밝힌 리액트의 존재이유

지속적으로 데이터가 변화하는 대규모 애플리케이션 구축



리액트의 특징

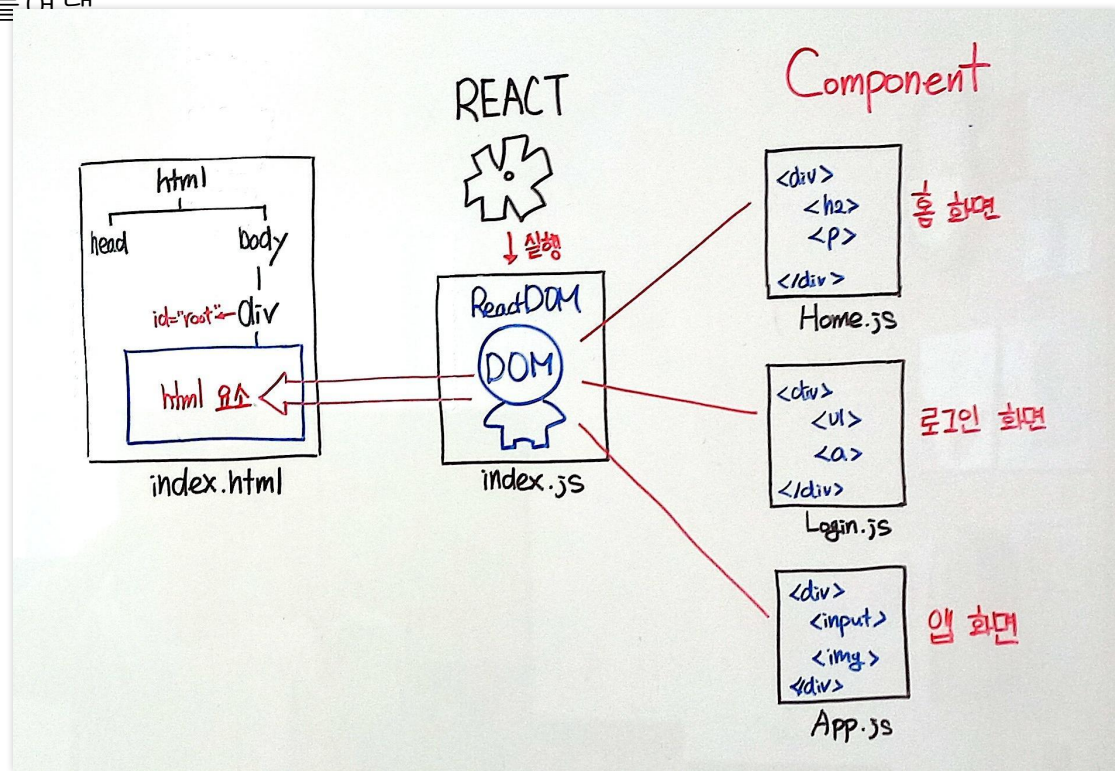
- **SPA** [Single Page Application]
 - **JSX문법** [선언적 방식]
 - `document.createElement('div')` 식으로 요소를 만드는 것을 [명령형 방식]이라고 부름
 - **컴포넌트 [Component]** - 재사용성
 - ReactDOM 이라는 가상돔을 통해 부드러운 화면전환이 가능
 - **state, props** 멤버 프로퍼티의 존재 이해
 - 라이브러리/프레임워크 모두 가능, ES6버전 완벽지원
- ❖ 동적인 웹 페이지를 보다 효율적으로 구현하기 위한 라이브러리 / 프레임워크

리액트의 SPA 와 JSX

- 기존에는 사용자 입력에 따라 매번 **html** 파일 다운로드
 - SPA에서는 **javascript**를 이용해 **html**의 필요한 부분만 업데이트하는 것이 가능
 - SPA에서는 코드 파일이 **Component**를 기준으로 작성
 - 그래서 **React**에서는 스타일 **CSS** 코드도 **Component** 파일 내부에 함께 작성할 수도 있음.
-
- **html** 파일을 여러개 만들지 않고.. **JS**를 이용하여 요소들을 만들어 내는 방식이 **React**의 특징.
 - **JS**를 이용하여 요소를 만들려면 코드가 다소 길고 복잡함.
 - **React**의 **JSX**는 **<>**태그문을 **JS**안에서 작성할 수 있도록 한 것임
 - **.jsx**에서 **<>**태그문으로 작성하면 **REACT**에서 **document.createElement()** 와 같은 코드로 변경해주는 방식임.
 - 결국 개발자는 **html**문서를 더이상 만들지 않고 **JS**문서 안에서 **<>**태그문을 작성하면 됨.
 - 다시말해 자바스크립트로 웹앱을 개발

리액트의 SPA 와 JSX - React DOM

- html 파일을 여러개 만들지 않고.. JS를 이용하여 ReactDOM 이라는 가상돔을 통해 화면을 동적으로 만들어내



리액트의 Component

- React를 사용하면 컴포넌트라 불리는 조각들을 사용하여 사용자 인터페이스를 만들 수 있음.
- React 앱은 컴포넌트로 구성
- 컴포넌트는 고유한 로직과 모양을 가진 사용자 인터페이스 UI의 일부
- 컴포넌트는 버튼만큼 작을 수도 있고 전체 페이지만큼 클 수도 있음.
- 컴포넌트는 함수형 컴포넌트 또는 클래스형 컴포넌트로 만들 수 있음.

함수형 컴포넌트

```
function Welcome(props) {  
  return <h1>Hello sam</h1>;  
}
```

클래스형 컴포넌트

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello sam</h1>;  
  }  
}
```

리액트의 Component

- React 컴포넌트는 마크업을 반환하는 자바스크립트 함수 또는 클래스

```
function MyButton() {  
  return (  
    <button>I'm a button</button>  
  );  
}
```

- 이제 MyButton을 선언했으므로 다른 컴포넌트 안에 중첩할 수 있음

```
function MyApp() {  
  return (  
    <div>  
      <h1>Welcome to my app</h1>  
      <MyButton />  
    </div>  
  );  
}
```

<MyButton />이 대문자로 시작하는 것을 주목

이것이 바로 **React** 컴포넌트임을 알 수 있는 방법

React 컴포넌트의 이름은 항상 대문자로 시작해야 하고 **HTML** 태그는 소문자로 시작

결과화면

Welcome to my app

I'm a button

JSX로 마크업 작성하기

- JSX는 HTML보다 엄격
- JSX에서는 `<br />` 같이 태그를 닫아야 함
- 또한 컴포넌트는 여러 개의 JSX 태그를 반환할 수 없음
- `<div>...</div>` 또는 빈 `<>...</>` 래퍼와 같이 공유되는 부모로 감싸야 함

```
function AboutPage() {  
  return (  
    <div>  
      <h1>About</h1>  
      <p>Hello there.</p>  
    </div>  
  );  
}
```

```
function AboutPage() {  
  return (  
    <>  
      <h1>About</h1>  
      <p>Hello there.</p>  
    </>  
  );  
}
```

데이터 표시하기

- JSX를 사용하면 자바스크립트에 마크업을 넣을 수 있음
- 중괄호 `{}`를 사용하면 태그문 코드에서 변수를 삽입하여 사용자에게 데이터를 표시할 수 있음
- 아래의 예시는 `user.name`을 표시

```
function UserPage() {  
  
  let user= {name:"same", age:20 }  
  
  return (  
    <h1>  
      {user.name}  
    </h1>  
  );  
}
```

데이터 표시하기

- 이미지의 경로를 넣을 수도 있음
- 스타일의 값을 JS변수로 지정할 수도 있음.

Hedy Lamarr



```
const user = {
  name: 'Hedy Lamarr',
  imageUrl: 'https://i.imgur.com/yXOvdOSs.jpg',
  imageSize: 90,
};

export default function Profile() {
  return (
    <>
      <h1>{user.name}</h1>
      <img
        className="avatar"
        src={user.imageUrl}
        alt={'Photo of ' + user.name}
        style={{
          width: user.imageSize,
          height: user.imageSize
        }}
      />
    </>
  );
}
```

스타일 추가하기

1. 별도의 CSS파일로 스타일 작성

- a. HTML에 <link> 태그를 추가
- b. JSX에서 `import` 를 통해 적용

1. JSX에서 요소의 속성으로 스타일객체 지정

❖ React는 CSS 파일을 추가하는 방법을 특별하게 규정하고 있지는 않음. 개발자의 선택에 따름.

스타일 추가하기 2. 별도의 CSS파일

- React에서는 `className`으로 CSS 클래스를 지정
- HTML의 `class` 어트리뷰트와 동일한 방식으로 동작
- 그 다음 별도의 CSS 파일에 해당 CSS 규칙을 작성

```
<img className="avatar" />
```

```
.avatar {  
  border-radius: 50%;  
}
```

CSS 파일 적용하기

a. HTML 파일에서 CSS파일

`<link>`

```
<link rel="stylesheet" href="./myStyle.css">
```

b. JS 파일에서 CSS파일 import

```
import "./myStyle.css"
```

스타일 추가하기 2. style 속성값으로 스타일객체 지정

- `style={{}}`은 특별한 문법이 아니라 `style={}` JSX 중괄호 안에 있는 일반 `{}` 객체임.
- 스타일이 자바스크립트 변수에 의존하는 경우 `style` 속성을 사용할 수 있음.

```
<img
  className="avatar"
  src={user.imageUrl}
  alt={'Photo of ' + user.name}
  style={{
    width: user.imageSize,
    height: user.imageSize
  }}
/>
```

Hedy Lamarr



조건부 렌더링

- if 문 또는 논리연산을 사용하여 조건부로 **JSX**를 포함할 수 있습니다.

if 문 분기

```
let content;
if (isLoggedIn) {
  content = <AdminPanel />;
} else {
  content = <LoginForm />;
}
return (
  <div>
    {content}
  </div>
);
```

삼항연산자를 이용한 분기

```
<div>
  {isLoggedIn ? (
    <AdminPanel />
  ) : (
    <LoginForm />
  )}
</div>
```

else 분기가 필요하지 않다면 && 연산자로 조건부 렌더링

```
<div>
  {isLoggedIn && <AdminPanel />}
</div>
```

리스트 렌더링

- 컴포넌트 리스트를 렌더링하기 위해서는 for 문 또는 map() 함수와 같은 자바스크립트 기능을 사용

```
const products = [
  { title: 'Cabbage', id: 1 },
  { title: 'Garlic', id: 2 },
  { title: 'Apple', id: 3 },
];

const listItems = products.map(product =>
  <li key={product.id}>
    {product.title}
  </li>
);

return (
  <ul>{listItems}</ul>
);
```

- Cabbage
- Garlic
- Apple

이벤트 처리

- 컴포넌트 내부에 *이벤트 핸들러* 함수를 선언하여 이벤트에 응답

```
function handleClick() {  
  alert('You clicked me!');  
}  
  
return (  
  <button onClick={handleClick}>  
    Click me  
  </button>  
);
```

onClick={handleClick}의 끝에 소괄호()가 없는 것을 주목

이벤트 핸들러 함수를 호출하지 않고 **전달**만 하면 됨.

React는 사용자가 버튼을 클릭할 때 이벤트 핸들러를 호출

state (상태)

- 컴포넌트에서 화면에 보여지며 **변경 가능한 데이터**
- 사용자 인터랙션, 네트워크 응답 등에 따라 값이 변할 수 있음
- 컴포넌트가 **동적으로 UI를 업데이트** 할 수 있도록 함
- 컴포넌트 내부에서 선언하고 관리하는 아주 특별한 멤버변수
- 값이 바뀌면 컴포넌트가 **자동으로 리렌더링** 됨
- 클래스형 컴포넌트에서는 `this.state`와 `this.setState()` 사용
- 함수형 컴포넌트에서는 React Hook (`useState`)을 자주 사용

State는 컴포넌트 내부에서 관리하며, 값이 변할 때마다 **UI를 다시 그림**

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  handleClick = () => {
    this.setState({count: this.state.count + 1 });
  }

  render() {
    return (
      <div>
        <p>{this.state.count}번 클릭됨</p>
        <button
          onClick={this.handleClick}>클릭</button>
        </div>
      );
    }
  }
```

props (속성)

- "Properties"의 줄임말
- 부모 컴포넌트가 자식 컴포넌트에 값을 전달할 때 사용
- 읽기 전용 (immutable)
- 컴포넌트 간 데이터 전달에 사용
- 컴포넌트를 재사용 가능하게 만들
- 자식 컴포넌트는 props를 직접 수정할 수 없음
- 클래스형 컴포넌트에서는 `this.props` 사용
- 함수형 컴포넌트에서는 함수의 파라미터 (`props`) 를 사용

Props는 부모 → 자식 방향으로 데이터를 전달하는 읽기 전용 값

```
import React from 'react';
```

//클래스형 컴포넌트

```
class Welcome extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}!</h1>;  
  }  
}
```

// 사용 예

```
<Welcome name="Alice" />
```

// 함수형 컴포넌트

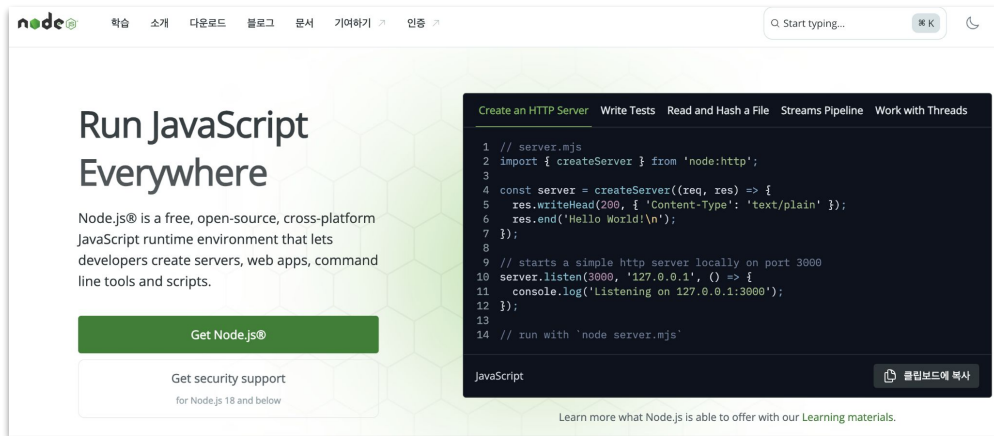
```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

리액트 개발

- **Node.js + npm** 설치하기 [**yarn** 설치로 대체 가능]
- 코드편집기 설치하기 - **Visual Studio code**
- **Git** 설치
- **CRA(create-react-app) or Vite** 프레임워크로 **React** 프로젝트 만들기 [현재는 Vite를 권장 ~ 발음 : 비트]

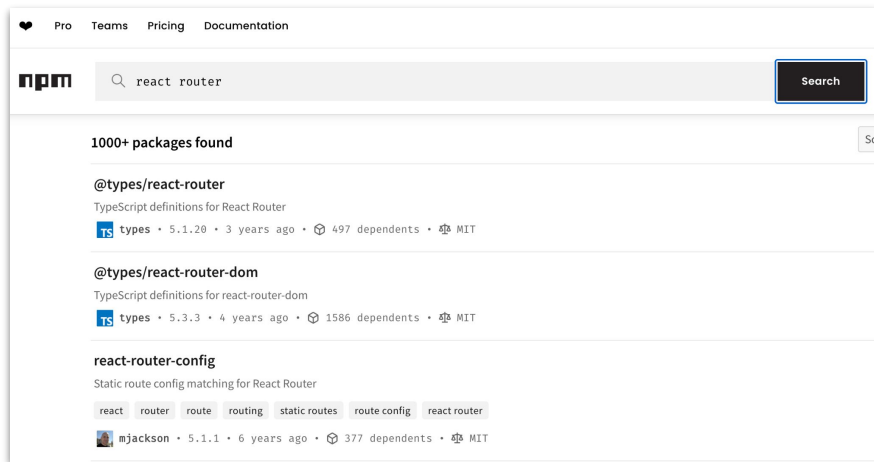
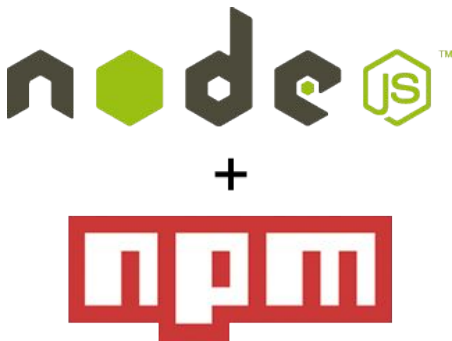
Node.js

- 리액트 프로젝트를 만들려면 **Node.js** 를 반드시 설치해야 함.
- 크롬 V8 자바스크립트 엔진으로서 자바스크립트 런타임. (즉, 자바스크립트를 해석하는 인터프리터)
- 웹 브라우저 환경이 아닌 곳에서도 자바스크립트를 사용할 수 있도록 해줌.
- 웹 서버(BackEnd), 웹 애플리케이션, 모바일 앱, 데스크탑 애플리케이션 등 모든 영역에서 자바스크립트 사용



Node.js 와 npm

- 리액트 애플리케이션은 웹 브라우저에서 실행되기에 **Node.js** 가 없어도 무방
- 하지만, 리액트 프로젝트를 빌드하는데 사용하는 주요 개발도구인 **Babel(바벨)**, **Webpack(웹팩)**에 필요
- Node.js를 설치하면 패키지 매니저 도구인 **npm**이 같이 설치됨.
- **npm(node package manager)** : 자바스크립트로 만든 수많은 패키지(라이브러리)를 설치 및 관리해주는 도구



Babel js

- 바벨(Babel)은 자바스크립트 코드를 변환해주는 도구(라이브러리)
- 최신 자바스크립트 문법을 이전 버전과 호환되도록 변환하여 구형 브라우저에서도 동작하게 만들 수 있음

자바스크립트 컴파일러:

자바스크립트 코드를 입력받아 다른 버전의 자바스크립트 코드로 변환하는 컴파일러

최신 문법 지원:

최신 자바스크립트 문법(ES2015+, ESNext)을 사용하면 코드를 더 간결하고 효율적으로 작성할 수 있음.

하위 호환성:

최신 문법을 구형 브라우저에서도 지원하는 이전 버전의 자바스크립트 코드로 변환하여, 호환성 문제를 해결.

다양한 기능:

플러그인 및 프리셋을 통해 다양한 기능을 제공

예) TypeScript를 자바스크립트로 변환하거나, JSX 코드를 변환

Node.js 와 npm 설치하기

windows 기준

- 공식 홈페이지에서 GUI 설치 프로그램 다운로드

mac os 기준

- 방법1. Node Version Manager (nvm) 을 이용한 설치
- 방법2. Homebrew 를 이용한 설치
- 방법3. 공식 홈페이지에서 GUI 설치 프로그램 다운로드 (.pkg 파일)

방법 1. nvm(Node Version Manager) 사용 [추천]

- 여러 버전의 Node.js를 쉽게 설치하고 전환할 수 있어서 개발에 가장 적합

1. 터미널 열기

2. nvm 설치

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
```

3. 설치 완료 후 환경변수 설정 확인

설치 후 터미널을 재시작한 후 아래 명령어로 nvm을 사용할 수 있는지 확인:

```
nvm
```

4. Node.js 설치

최신 LTS 버전 설치:

특정 버전(v22) 설치 :

```
nvm install --lts or nvm install 22
```

5. Node 버전 확인

```
node -v or node --version
```

6. npm 버전 확인

```
npm -v or npm --version
```

만약 nvm 명령을 인식하지 못하면,
.zshrc 또는 .bash_profile에 다음 줄이 있는지 확인

```
export NVM_DIR="$HOME/.nvm"  
[ -s "$NVM_DIR/nvm.sh" ] && \.  
"$NVM_DIR/nvm.sh"
```

방법2. Homebrew 사용

- 일반적으로 mac os 에서 패키지를 설치하는 가장 기본적인 CLI 도구 (간단하지만 버전 관리가

1. 터미널 열기

2. Homebrew 설치 (mac os에 설치되어 제공되기도 함)

```
curl -o- https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh | bash
```

3. 설치 완료 후 환경변수 설정 확인

설치 후 터미널을 재시작한 후 아래 명령어로 brew을 사용할 수 있는지 확인:

```
brew
```

4. Node.js 설치

```
brew install node or brew install node@22
```

5. Node 버전 확인

```
node -v or node --version
```

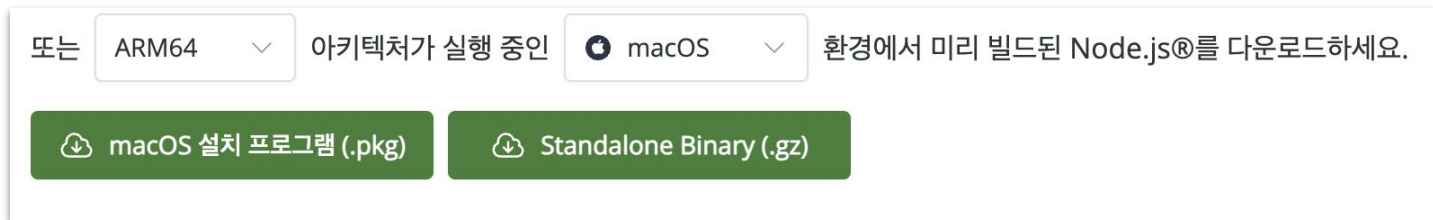
6. npm 버전 확인

```
npm -v or npm --version
```

만약 **brew** 명령을 인식하지 못하면,
설치 메시지의 마지막 명령어를 참고하여
.zshrc 또는 .bash_profile에 homebrew 의 path
지정

방법3. 공식 웹사이트에서 직접 설치 (GUI 설치 프로그램)

- CLI에 익숙하지 않다면 GUI 설치도 가능



리액트 프로젝트 만들기

- **CRA** create-react-app

- `npx create-react-app` 프로젝트명 [프로젝트명은 소문자만 허용]
- `yarn create react-app` 프로젝트명 [`yarn` : `npm`과 같은 역할의 패키지설치 프로그램. (페이스북에서 개발. 속도빠름)]

- **Vite** [CSR지원] - 추천

- `npm create vite@latest` 프로젝트명 (소문자 영어만 가능, _랑 - 가능)
- `yarn create vite@latest` 프로젝트명

- **Next.js** [SSR지원]

- `npx create-next-app` 프로젝트명
- `yarn create next-app` 프로젝트명

Vite 프로젝트 폴더 및 파일 구조

파일/폴더

역할

`public/`

빌드 과정 없이 그대로 배포되는 정적 파일(이미지, `favicon` 등)을 저장

`src/`

실제 **React** 애플리케이션 소스 코드가 들어있는 폴더

`src/assets/`

컴포넌트에서 사용하는 이미지, 아이콘, 폰트 등의 리소스 저장

`src/App.jsx`

메인 **React** 컴포넌트(화면 UI의 시작점)

`src/main.jsx`

React 앱을 **DOM**에 연결하고 최초 렌더링하는 진입점

`src/index.css`

전역 스타일(**CSS**) 정의

`package.json`

프로젝트 정보, 의존성 라이브러리, 실행 스크립트 관리

`vite.config.js`

Vite 개발 서버 및 빌드 설정

`index.html`

브라우저가 처음 로드하는 **HTML** 파일

`node_modules/`

설치된 **npm** 패키지들이 저장되는 폴더(자동 생성)

`package-lock.json`

의존성 버전을 고정하여 동일한 환경을 보장

`.gitignore`

Git이 추적하지 않을 파일/폴더 목록

package.json 과 package-lock.json 차이

package.json

- 프로젝트의 **메타데이터**와 **의존성 목록**을 정의
- React, React DOM, 기타 라이브러리를 어떤 버전으로 사용할지 지정
- 스크립트(`npm start`, `npm run build` 등), 프로젝트 이름, 버전, 라이선스 등도 포함

```
{
  "name": "my-react-app",
  "version": "1.0.0",
  "scripts": {
    "start": "react-scripts start"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  }
}
```

- ❖ 개발자가 직접 수정하는 파일
- ❖ Git 등의 버전관리 시스템에 포함
- ❖ 다른 개발자가 프로젝트를 설치할 때는 이 파일을 기준으로 필요한 패키지를 받음

package-lock.json

- **package.json**에 명시된 의존성들을 **정확한 버전까지 고정**해서 기록
- 하위 의존성(의존성의 의존성들)까지 **모든 버전 정보**를 포함
- **npm install**을 실행할 때 항상 **동일한 버전 트리**를 **재현**할 수 있도록 도와줌

```
"node_modules/@adobe/css-tools": {
  "version": "4.4.3",
  "resolved":
    "https://registry.npmjs.org/@adobe/css-tools/-/css-tools-
    4.4.3.tgz",
  "integrity":
    "sha512-VQKMkwriZbaOgVCby1UDY/LDk5fljhQicCvVP
    Fqfe+69fWaPWYdbWJ3wRt59/Yzlwda1l81loas3oCoHxn
    qvdA==",
  "license": "MIT"
},
```

- ❖ 자동 생성되는 파일 (**npm install** 실행 시).
- ❖ **수동으로 수정하지 않아야 함**
- ❖ 다른 개발자가 설치할 때도 **완전히 동일한 환경**이 만들어지도록 함
- ❖ **Git**에 반드시 포함해야 하는 파일